

Department of
Electrical & Electronics Engineering

Abdullah Gül University

BIOMEDICAL SYSTEM DESIGN CAPSULE-S1

Submitted on: 25.10.2026

Submitted by: Enes Erdoğan 2111011061

Grade: / 100

Part A — Dataset Selection

A.1 Dataset 1 (linear regression): Diabetes

Problem and source. For the first part I used the Diabetes dataset from Efron, Hastie, Johnstone and Tibshirani (2004). It comes with scikit-learn, and I loaded it with *load_diabetes(scaled=False)* so that the values stay in their real clinical units. There are 442 diabetes patients and for each patient ten measurements were taken at the start. The target is a number that shows how much the disease progressed after one year (range 25–346, mean 152.1, std 77.1). My task was to predict this progression score.

Features and target. The inputs are age, sex, bmi, bp and six blood-serum measurements s1–s6 (Table 1). The target is disease progression, which is a continuous value.

Var.	Description	Mean	Std	Min	Max	r(y)
age	Age (years)	48.52	13.11	19.00	79.00	0.19
sex	Sex (coded 1 / 2)	1.47	0.50	1.00	2.00	0.04
bmi	Body-mass index (kg/m²)	26.38	4.42	18.00	42.20	0.59
bp	Average blood pressure (mm Hg)	94.65	13.83	62.00	133.00	0.44
s1	tc – total serum cholesterol	189.14	34.61	97.00	301.00	0.21
s2	ldl – low-density lipoproteins	115.44	30.41	41.60	242.40	0.17
s3	hdl – high-density lipoproteins	49.79	12.93	22.00	99.00	-0.40
s4	tch – total cholesterol / HDL	4.07	1.29	2.00	9.09	0.43
s5	ltg – log of serum triglycerides	4.64	0.52	3.26	6.11	0.57
s6	glu – blood sugar level	91.26	11.50	58.00	124.00	0.38
target	Disease progression one year after baseline	152.13	77.09	25.00	346.00	—

Table 1. Diabetes dataset – variables, summary statistics and Pearson correlation r(y) with the target.

Why it is suitable. I chose this dataset because the target is continuous and some features have a clear and almost linear relation with it (bmi r = 0.59, s5 r = 0.57). It also has more than 100 samples and more than two features, so it meets the requirements. Another reason is that the features have very different scales (sex ≈ 1–2, s1 ≈ 97–301), so I thought it would be a good dataset for the scaling experiment. Also some features are strongly correlated with each other (s1 and s2), and this helped me when I discussed the meaning of the coefficients.

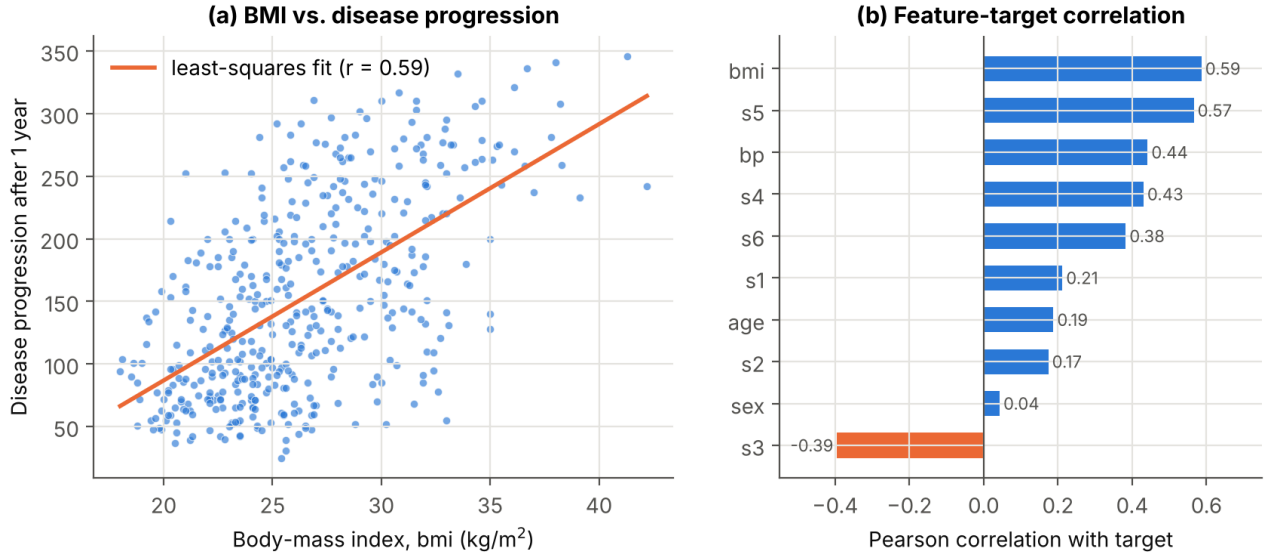


Figure 1. (a) BMI against disease progression with a least-squares line: higher BMI goes together with faster progression, but the points are spread a lot. (b) Correlation of each feature with the target; s3 (HDL, "good" cholesterol) is the only clearly negative one.

Preprocessing. I checked for missing values and there were none (0 missing cells). I did not remove any outliers, because all values look possible from a medical point of view. I split the data 80/20 (353 train / 89 test, random_state = 42). For the scaling and gradient descent experiments I standardised the features ($z = (x - \mu)/\sigma$). I calculated μ and σ only on the training set, so no information from the test set leaks into the model.

A.2 Dataset 2 (logistic regression): Breast Cancer Wisconsin (Diagnostic)

Problem and source. For the second part I used the Breast Cancer Wisconsin (Diagnostic) dataset from the UCI Machine Learning Repository (Wolberg, Street and Mangasarian, 1995). It is also included in scikit-learn (*load_breast_cancer*). There are 569 samples, and each one describes the cell nuclei in a digital image of a fine-needle aspirate of a breast mass, with 30 numeric features. The task is to classify the tumour as malignant or benign.

Features and target. The target is *malignant* = 1 (212 samples, 37 %) and benign = 0 (357, 63 %). In scikit-learn the original coding is the opposite (0 = malignant), so I inverted it. This way the positive class is the disease, and precision and recall have a meaningful medical interpretation. For the logistic experiments I selected five "mean" features that are easy to interpret (Table 2).

Feature	Description	Mean	Std	Min	Max
mean radius	Mean distance from centre to perimeter of nuclei	14.127	3.524	6.981	28.110
mean texture	Std. dev. of grey-scale values	19.290	4.301	9.710	39.280
mean smoothness	Local variation in radius lengths	0.096	0.014	0.053	0.163
mean concave points	Number of concave portions of the contour	0.049	0.039	0.000	0.201
mean symmetry	Symmetry of the nucleus	0.181	0.027	0.106	0.304

Table 2. Selected features of the Breast Cancer dataset (all 569 samples).

Why it is suitable. The target is binary and the classes are a bit imbalanced, but not too much. In Figure 2 we can see that the two classes can mostly be separated with a straight line, but not perfectly. This is exactly the type of problem logistic regression is made for. The overlapping area also gives some realistic errors that I could analyse later.

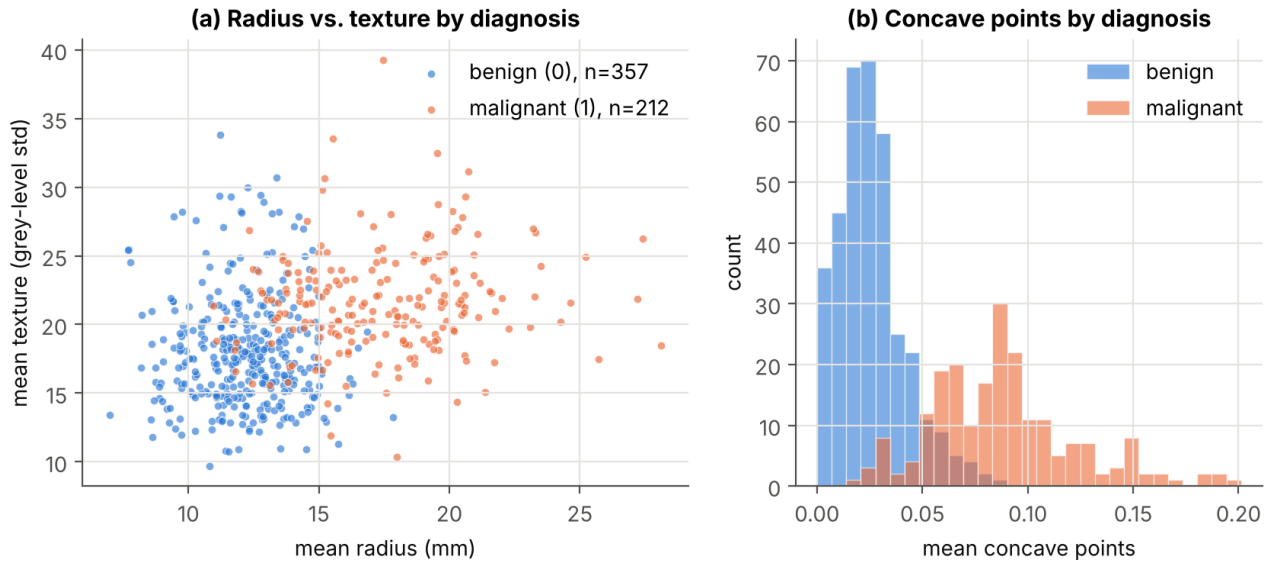


Figure 2. (a) Mean radius vs. mean texture coloured by diagnosis: malignant tumours are usually larger and have higher texture, and the classes overlap at around radius 13–16 mm. (b) Distribution of mean concave points: the two classes are separated well, so I expected this feature to get a large coefficient.

Preprocessing. There were no missing values (0). I used a stratified 80/20 split (455 train / 114 test; 42 malignant in test), so both sets keep the same 37/63 class ratio. I standardised the features inside a scikit-learn Pipeline, so the scaler is fitted only on the training data. I did this because the raw features are very different in size (radius ≈ 14 but smoothness ≈ 0.1), and L2 regularisation penalises all coefficients in the same way.

Part B — Linear Regression

Experiment B1 — Model parameters and predictions

First I trained a basic ordinary least squares model $\hat{y} = b + \Sigma\, wjxj$ on the raw (unscaled) features. The coefficients are shown in Table 4 (column "raw units") and the error metrics are in Table 3.

Model / data	MAE	MSE	RMSE	R^2
Training set	43.48	2868.55	53.56	0.528
Test set	42.79	2900.19	53.85	0.453
Test set – mean-only baseline	64.01	5361.53	73.22	-0.012

Table 3. B1 error metrics. The mean-only baseline always predicts the training mean.

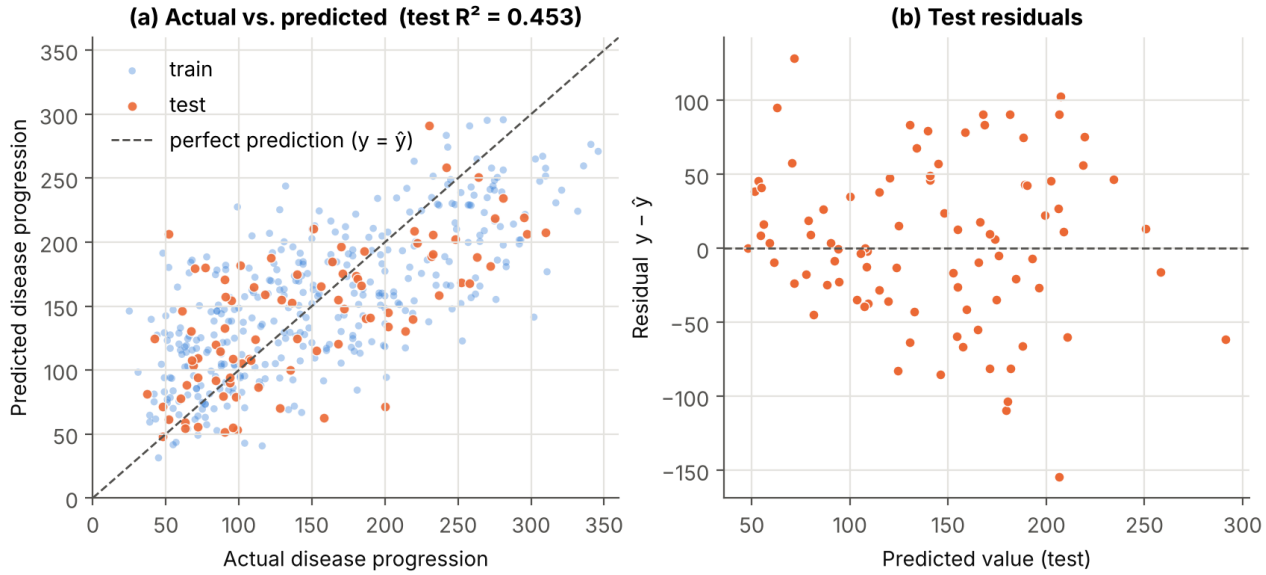


Figure 3. B1 – (a) actual vs. predicted progression for training (light) and test (dark) points; a perfect model would put all points on the dashed line. (b) Test residuals: there is no strong pattern, but the spread gets bigger for larger predictions and high actual values are predicted too low.

1. What do the coefficients and intercept represent? A coefficient shows how much the predicted disease progression changes when that feature increases by one unit and all the other features stay the same. For example, wbmi = 5.85, so one extra kg/m² of BMI adds about 5.8 points to the progression. wbp = 1.20 per mm Hg, and ws5 = 67.1 per unit of log-triglycerides. wsex = -23.1 means that, if all other values are equal, the group coded as 2 has about 23 points less progression than group 1. The intercept (-341.4) is the prediction when all features are zero. This point is outside the data (nobody has BMI = 0 or blood pressure = 0), so I think it has no real medical meaning. It is only needed to place the regression plane in the right position.

2. How do the features influence predictions? We cannot compare the raw coefficients directly, because the features have different units. So I looked at the standardised coefficients (B2, Table 4). Per standard deviation, the most important features are s1 (-44.4), s5 (35.2), bmi (25.6), s2 (24.6) and bp (16.8). Age and s6 have almost no effect when the other features are already in the model. One interesting thing is that s1 has a positive correlation with the target (r = 0.21) but a large negative coefficient. The reason is that s1 (total cholesterol) and s2 (LDL) are strongly correlated, so the model actually uses the difference between them. From this I understood that coefficients show partial effects, not simple correlations.

3. How accurately does it predict unseen data? On the test set I got $R^2 = 0.453$, so the model explains about 45 % of the variance in disease progression. The typical error is MAE ≈ 43 points (RMSE ≈ 54), while the standard deviation of the target is 77. This is clearly better than the mean-only baseline (MAE 64, $R^2 \approx 0$), but it is not very accurate. Training and test results are close (R^2 0.528 vs. 0.453), so I don't think the model is overfitting. I think the limit comes from the data itself: ten measurements are not enough information, and there are also non-linear effects. In Figure 3b we can see that patients with very high progression are always predicted too low.

Experiment B2 — Feature scaling

I trained the OLS model two times: once on the raw features and once on the standardised features (scaler fitted on the training set). I also wanted to show how scaling affects the *training process*, so I trained the same linear model with batch gradient descent (GD) for 5000 iterations in both cases. For each case I used the largest learning rate that was still stable.

Feature	Coefficient – raw units	Coefficient – standardised	Training std. dev.
age	0.138	1.75	12.74
sex	-23.064	-11.51	0.50
bmi	5.846	25.61	4.38
bp	1.197	16.83	14.06
s1	-1.282	-44.45	34.68
s2	0.811	24.64	30.38
s3	0.602	7.68	12.76
s4	10.160	13.14	1.29
s5	67.109	35.16	0.52
s6	0.202	2.35	11.66
Intercept	-341.38	153.74	—

Table 4. OLS coefficients before and after standardisation. Standardised coefficient = raw coefficient × training standard deviation of the feature (e.g. bmi: 5.85 × 4.38 ≈ 25.6).

Condition	Test MAE	Test MSE	Test R^2	Final train MSE
OLS, no scaling	42.79	2900.19	0.453	—
OLS, standardised	42.79	2900.19	0.453	—
Grad. descent, no scaling (lr = 2.6e-05, 5000 it.)	44.55	3053.37	0.424	3221.46
Grad. descent, standardised (lr = 0.1, 5000 it.)	42.79	2899.86	0.453	2868.55

Table 5. Test performance with and without scaling. The OLS optimum training MSE is 2868.55.

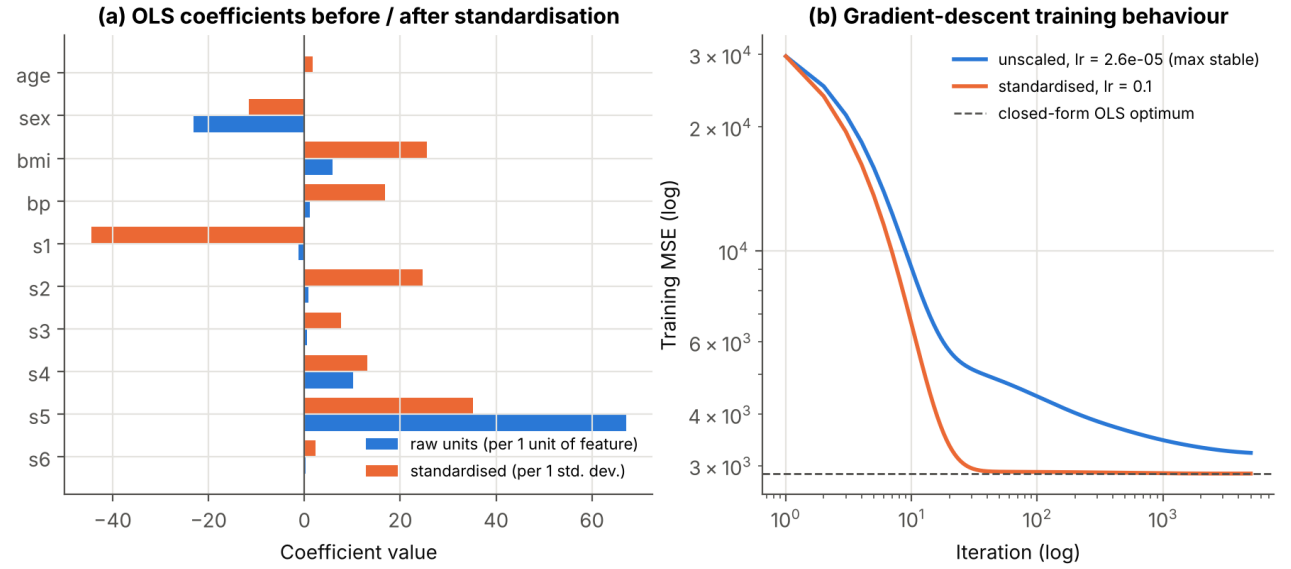


Figure 4. B2 – (a) coefficients in raw units and after standardisation; (b) training loss of gradient descent with and without scaling. Without scaling GD is still far from the optimum after 5000 iterations, but with scaling it reaches the optimum after about 60 iterations.

1. How do the coefficients change? Each coefficient gets multiplied by the standard deviation of its feature. The intercept becomes the training mean of the target (153.7), because all scaled features now have mean zero. After scaling, the coefficients can be compared with each other. For example, s5 goes from 67.1 to 35.2 (because s5 has a small std of 0.52), but s1 goes from -1.28 to -44.4 (because s1 has a large std of 34.6). So the order of importance changes completely, but none of the signs change.

2. Does scaling change prediction accuracy? For the closed-form OLS solution, no. The biggest difference between the test predictions of the two models was 7.1e-13, which is just numerical round-off error. MAE, MSE and R^2 are exactly the same. This makes sense, because standardisation is an invertible linear transformation and a linear model without regularisation can simply absorb it into its weights. But for gradient descent it *does* make a difference in practice: after the same 5000 iterations, unscaled GD reached test R^2 0.424, while scaled GD reached 0.453.

3. Why is scaling important? How fast gradient descent works depends on the shape of the loss surface. Without scaling, the Hessian $(1/n)X^TX$ has a largest eigenvalue of $\lambda max \approx 73491$ and a condition number of ≈ 5.0e+07. This means the surface looks like a very long and narrow valley. For stable training the step size must be smaller than $2/\lambda max \approx 2.7e\text{-}05$. When I tried a "normal" learning rate of lr = 10^{-4}, it diverged after 6 iterations. But with such a tiny step, the progress in the flat directions is extremely slow. After standardisation $\lambda max \approx 3.97$ and the condition number goes down to ≈ 430, so one learning rate works well for all directions. In general, scaling is also very important for regularised models (L1/L2 penalties treat all coefficients the same), for distance-based methods (k-NN, SVM, k-means) and when we want to compare coefficient sizes.

Experiment B3 — Learning rate and convergence

I ran batch gradient descent on the standardised training data with five learning rates (0.001, 0.01, 0.1, 0.5, 1.0). I started from zero weights and used 2000 iterations, and I kept everything else the same. The theoretical stability limit is $2/\lambda max = 0.504$.

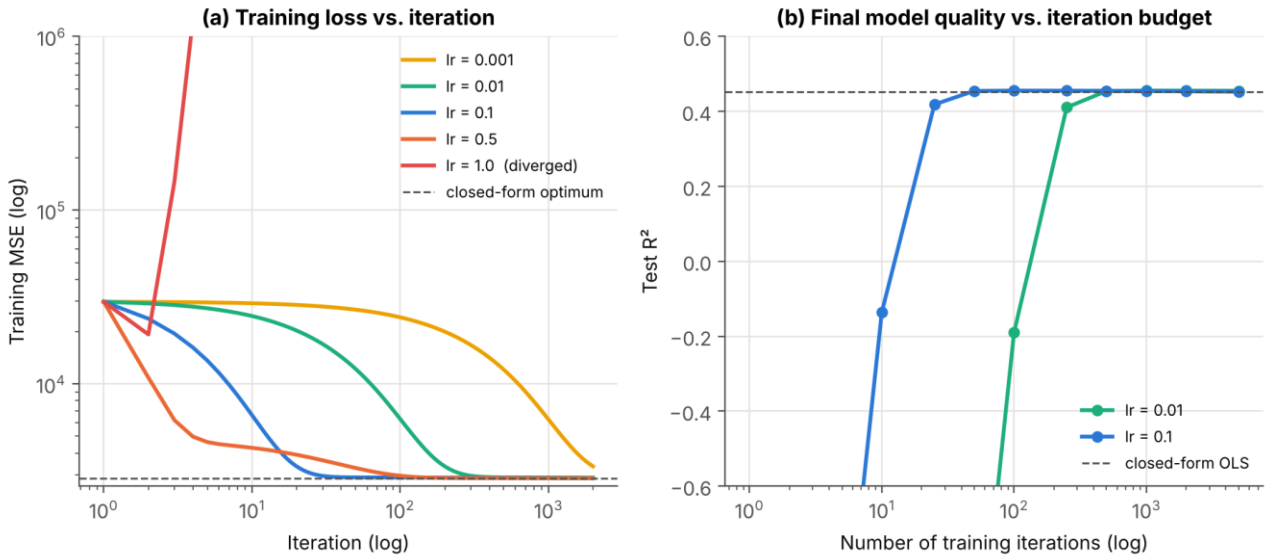


Figure 5. B3 – (a) training MSE vs. iteration for five learning rates (log–log axes); lr = 1.0 is above the stability limit and diverges. (b) Test R^2 of the model after a given number of iterations for lr = 0.01 and lr = 0.1.

Learning rate	Behaviour	Final train MSE	Iterations to ≤1 % of optimum	Test R^2
0.001	Not converged	3356.53	> 2000	0.349
0.01	Converged	2890.31	580	0.455
0.1	Converged	2869.33	58	0.454
0.5	Converged	2868.55	145	0.453
1.0	Diverged (loss → ∞)	—	—	—

Table 6. Learning-rate comparison after 2000 iterations.

Iterations	Train MSE (lr 0.01)	Test R^2 (lr 0.01)	Train MSE (lr 0.1)	Test R^2 (lr 0.1)	s1 coef. (lr 0.1)
10	24115.24	-3.150	5915.00	-0.136	-0.90
50	12026.72	-1.201	2898.23	0.455	-4.75
100	6224.03	-0.190	2894.74	0.455	-6.53
500	2898.68	0.454	2881.05	0.455	-18.26
2000	2890.31	0.455	2869.33	0.454	-37.90
5000	2881.05	0.455	2868.55	0.453	-44.04

Closed form	2868.55	0.453	2868.55	0.453	-44.45

Table 7. Effect of the number of iterations (standardised data). The last column shows the s1 coefficient, which converges very slowly.

1. Fastest stable convergence. lr = 0.1 reached the optimum fastest: it needed 58 iterations to get within 1 % of the optimal training MSE. lr = 0.5 is bigger, but it needed 145 iterations. The reason is that it is just under the stability limit (0.504), so at the beginning the loss jumps and the weights go back and forth along the steep direction before they settle down. This can be seen as the kink in Figure 5a. However, after 2000 iterations it ends up with the closest final value.

2. Too small / too large. With lr = 0.001 the loss goes down all the time, but very slowly. After 2000 iterations the training MSE was still 3357 and the test R^2 was only 0.349, so the model is under-trained. With lr = 1.0 every step jumps over the minimum by more than it corrects, so the error grows very fast and the loss explodes after a few iterations (divergence).

3. Number of iterations. If there are too few iterations the model is bad (lr = 0.1 with 10 iterations gives test R^2 -0.136). The test error becomes stable after about 50 iterations for lr = 0.1 and about 500 for lr = 0.01. After that, the training loss still goes down slowly, and the coefficients of the correlated pair s1/s2 keep changing (s1: -18.3 at 500 iterations, -44.0 at 5000, -44.4 in the closed-form solution). Something I found interesting is that the models that were not fully trained have a slightly *higher* test R^2 (0.455) than the exact solution (0.453). I think stopping early works like a kind of implicit regularisation (I discuss this more in Part D).

Part C — Logistic Regression

Experiment C1 — Model parameters and classification

I trained a logistic regression model $P(y = 1 \mid x) = \sigma(b + \Sigma\, w_j x_j)$, $\sigma(z) = 1/(1 + e^{-z})$, on the five standardised features from Table 2, and I evaluated it with a threshold of 0.5.

Feature	β (standardised)	Odds ratio e^{β} per 1 SD	β (raw units)	Training SD
mean radius	2.678	14.56	0.749	3.5751
mean texture	1.420	4.14	0.331	4.2859
mean smoothness	0.718	2.05	50.219	0.0143
mean concave points	2.263	9.62	57.097	0.0396
mean symmetry	0.387	1.47	14.019	0.0276
Intercept	-0.659	—	-27.866	—

Table 8. Logistic regression coefficients. Raw-unit θ = standardised θ / training SD.

Setting	Acc.	Prec.	Recall	F1	TN	FP	FN	TP
Test set (t = 0.5)	0.939	0.907	0.929	0.918	68	4	3	39

Table 9. C1 test performance (n = 114). ROC-AUC = 0.982, log-loss = 0.168, training accuracy = 0.941.

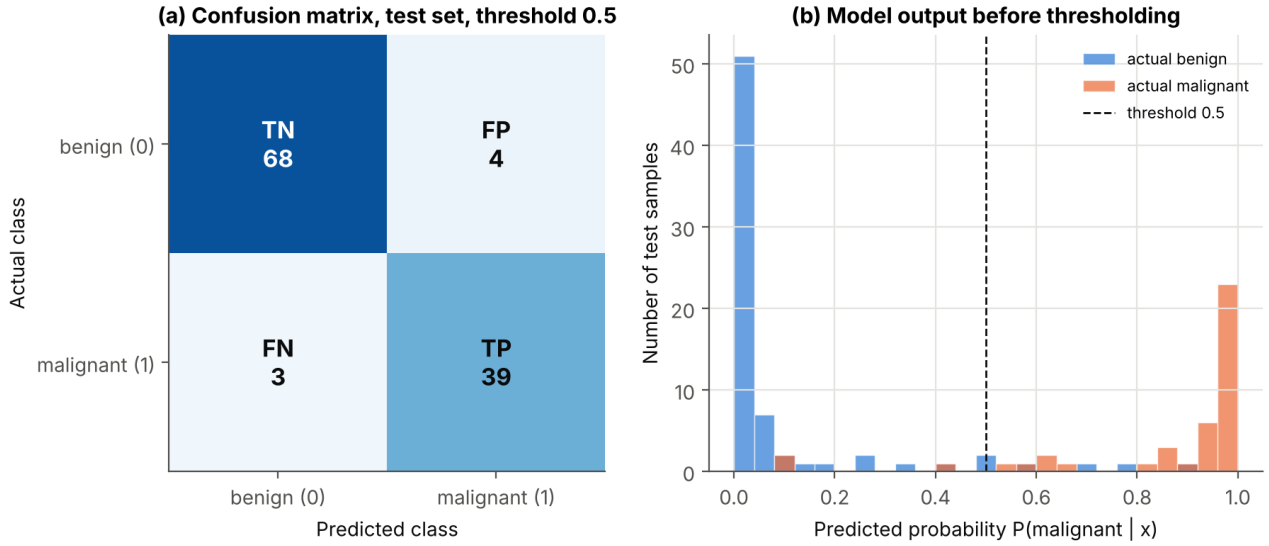


Figure 6. C1 – (a) confusion matrix on the test set at threshold 0.5. (b) Histogram of the predicted probabilities, so the model output before the threshold; most samples get probabilities close to 0 or 1, and the errors come from the few samples in the middle.

1. Output before the threshold. First the model calculates a linear score (log-odds) $z = b + w^T x$, which can be any real number. Then the sigmoid function turns it into a probability between 0 and 1. Here are three test patients as examples: z = -7.19 → p = 0.0008 (benign, correct); z = 11.25 → p = 0.99999 (malignant, correct); z = 0.40 → p = 0.60 (actually benign, but because it is a little above 0.5 it becomes a false positive). So the class label only comes after we apply the threshold.

2. Effect of coefficients. A coefficient shows how much the log-odds change when the feature increases by one unit, and e^{β} is the odds ratio. All five coefficients are positive, which means that larger, rougher, more concave and less symmetric nuclei all increase the probability of malignancy. Per standard deviation, mean radius multiplies the odds by 14.6, concave points by 9.6 and texture by 4.1, but symmetry (1.47) has only a small effect. The effect on the probability is not linear: the same change in z changes p a lot near p = 0.5, but almost nothing when p is already close to 0 or 1. The intercept is negative (-0.66), which means a tumour with average features is more likely to be benign ($\sigma(-0.66) \approx 0.34$, which is close to the 37 % prior).

3. Error types. The model made 4 false positives (benign tumours predicted as malignant) and 3 false negatives (malignant tumours that were missed). All 7 errors are "borderline" tumours: their mean radius is between 13.4 and 16.2 mm, which is exactly the overlap zone in Figure 2a. The false positives are benign tumours that are big or have high texture (for example radius 13.4 mm, texture 30.7). Two of the false negatives are malignant tumours with concave points at or below the dataset average of 0.049 (0.041 and 0.051), so with the features I used they look benign. In this application I think a false negative (missed cancer) is much worse than a false positive (an extra biopsy).

Experiment C2 — Decision threshold

I used the probabilities of the C1 model again without retraining, and applied thresholds of 0.3, 0.5 and 0.7. The test set should not be used to choose settings, so I *selected* the threshold using 5-fold cross-validated (out-of-fold) probabilities on the training set. The test results are only reported for evaluation.

Setting	Acc.	Prec.	Recall	F1	TN	FP	FN	TP
Train 5-fold CV, t = 0.3	0.923	0.865	0.941	0.901	260	25	10	160
Train 5-fold CV, t = 0.5	0.941	0.950	0.888	0.918	277	8	19	151
Train 5-fold CV, t = 0.7	0.938	0.986	0.847	0.911	283	2	26	144
Test set, t = 0.3	0.912	0.833	0.952	0.889	64	8	2	40
Test set, t = 0.5	0.939	0.907	0.929	0.918	68	4	3	39
Test set, t = 0.7	0.912	0.944	0.810	0.872	70	2	8	34

Table 10. Effect of the threshold (training cross-validation and test set).

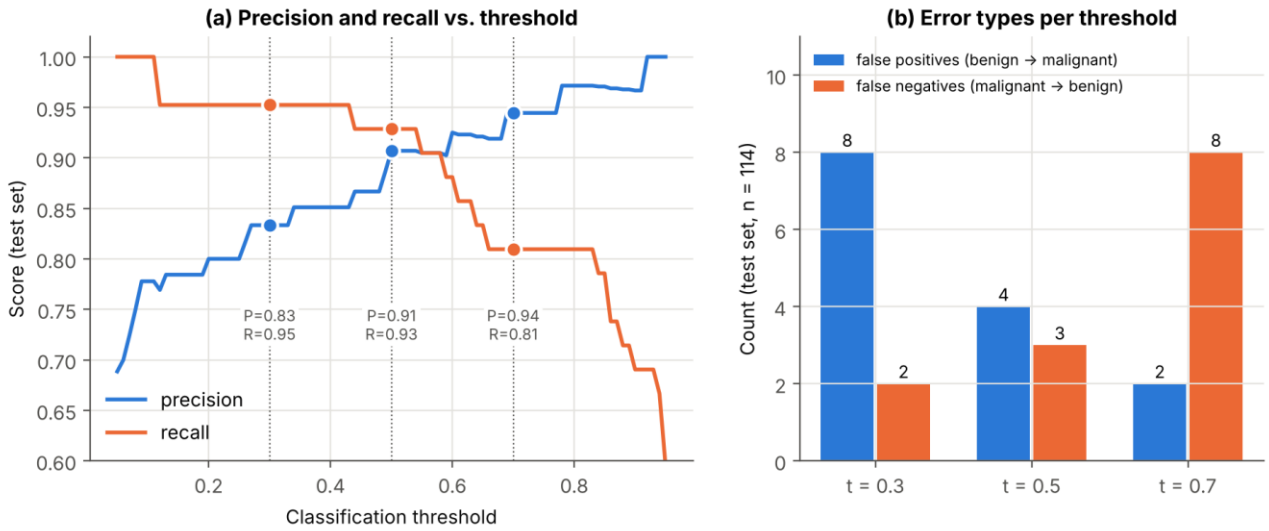


Figure 7. C2 – (a) precision and recall on the test set for every threshold from 0.05 to 0.95; the markers show 0.3, 0.5 and 0.7. (b) Number of false positives and false negatives at the three thresholds.

1. Effect on FP and FN. When I lower the threshold, more samples are labelled as malignant. At 0.3 the test set has 8 FP and 2 FN, at 0.5 it has 4 FP and 3 FN, and at 0.7 it has 2 FP and 8 FN. False positives and false negatives move in opposite directions. So the threshold only moves the errors from one type to the other, it does not make the model better at separating the classes.

2. Precision and recall. When the threshold goes up, precision increases (0.833 → 0.907 → 0.944) and recall decreases (0.952 → 0.929 → 0.810). The curves look like steps because the test set is small, so every step is just one sample crossing the threshold. Accuracy is highest at 0.5 and F1 does not change much, so these two metrics hide the trade-off that is important in medicine.

3. Selected threshold: 0.3. In cancer screening, missing a malignant tumour is much more dangerous than a false alarm, so recall is the most important metric for me. On the training cross-validation, 0.3 increases recall from 0.888 to 0.941 (missed cancers go from 19 to 10 out of 170), and the cost is 17 more false alarms. The test set also supports this choice: recall is 0.952 and only 2 out of 42 cancers are missed. I think a threshold of 0.7 would only make sense if false positives were the expensive error (for example before deciding on an invasive treatment).

Experiment C3 — Features and decision boundary

I trained two models on the same split: one with only *mean radius*, and one with *mean radius* and *mean texture*. I chose these two features because they are easy to visualise and they are only weakly correlated, so the second feature adds new information.

Model (test set, t = 0.5)	Acc.	Prec.	Recall	F1	ROC-AUC	Log-loss	FP	FN
1 feature: radius	0.860	0.882	0.714	0.789	0.937	0.293	4	12
2 features: radius + texture	0.868	0.886	0.738	0.805	0.955	0.258	4	11
Reference: 5 features (C1)	0.939	0.907	0.929	0.918	0.982	0.168	4	3

Table 11. One vs. two features (test set, threshold 0.5). The five-feature C1 model is shown for comparison.

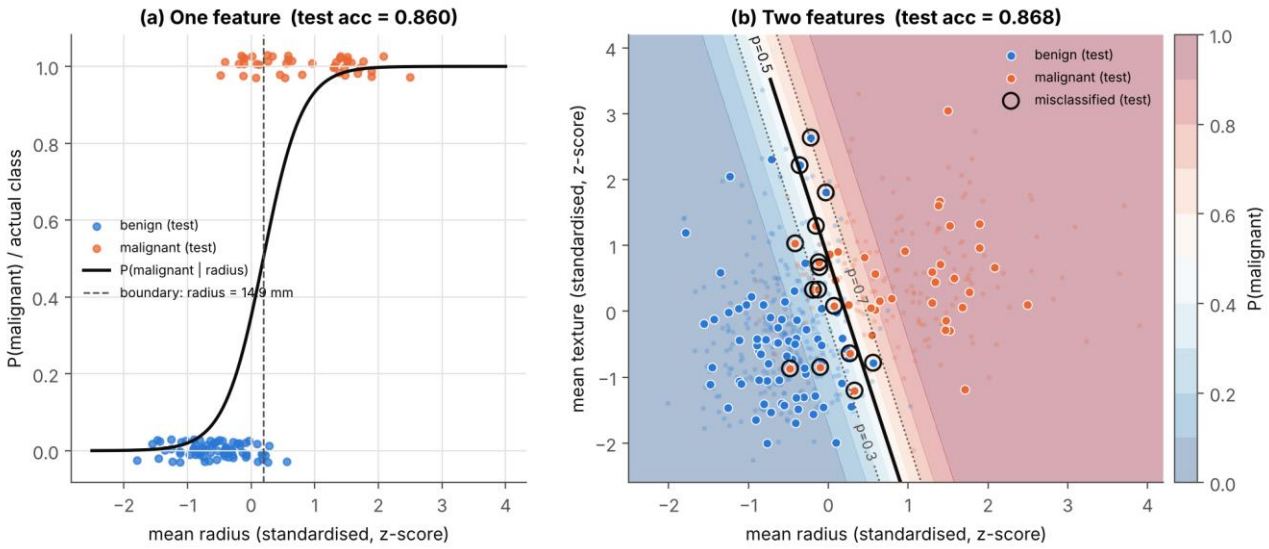


Figure 8. C3 – (a) the one-feature model is a sigmoid along the radius axis; its decision boundary is just one point, radius = 14.9 mm. (b) The two-feature model in standardised coordinates (z-scores): background colour = predicted probability, solid line = decision boundary p = 0.5, dotted lines = p = 0.3 and 0.7; small light points are training data, large points are test data, and circled points are misclassified.

1. Effect of the second feature. Adding texture made every metric better: accuracy 0.860 → 0.868, recall 0.714 → 0.738, ROC-AUC 0.937 → 0.955, log-loss 0.293 → 0.258. The accuracy gain is only one test sample, but the better AUC and log-loss show that the probabilities are ranked better and are more reliable. The texture coefficient (0.87) is much smaller than the radius coefficient (3.26), so texture is only a secondary signal. With five features (C1) the accuracy goes up to 0.939, mostly because of concave points.

2. What the boundary represents. The decision boundary is the set of points where the model cannot decide: P = 0.5, which means b + w1x1 + w2x2 = 0. In logistic regression it is always a straight line (or a hyperplane in more dimensions). The slope of the line shows the trade-off between the two features: a tumour with a smaller radius can still be classified as malignant if its texture is high enough. The p = 0.3 and p = 0.7 lines are parallel to it, so changing the threshold (C2) only moves the boundary, it does not rotate it.

3. Difficult observations. The samples close to the boundary are the hardest to classify. Out of the 35 test samples that are within 0.5 standard deviations of the line, 37 % are misclassified, but for the samples further away this is only 2.5 % (mean distance: 0.24 SD for errors and 0.98 SD for correct predictions). These are medium-sized tumours (radius ≈ 13–16 mm) where both classes overlap. Radius and texture just do not have enough information to separate them, and no straight line can do it. Most of the errors are small malignant tumours under the line (false negatives).

Part D — Reflection and Comparison

1. Linear vs. logistic regression output. Linear regression gives a real number with no limits, and this number is the prediction itself (for example a progression score of 180; in B1 my predictions were between about 48 and 291). Logistic regression also calculates a linear score, but then it puts it through a sigmoid to get a probability between 0 and 1 (Figure 6b), and the class label only comes after a threshold. Linear regression is trained by minimising squared error, and logistic regression by minimising log-loss (maximum likelihood). In both models the coefficients are “linear”, but in logistic regression they affect the log-odds, not the output directly.

2. Parameter vs. hyperparameter. Parameters are learned from the data during training. These are the weights and the intercept (for example wbmi = 5.85, b = -341.4 in B1; βradius = 2.68 in C1). Hyperparameters are chosen by me before training, and they control how the learning happens: the learning rate (0.001–1.0 in B3), the number of iterations, whether to standardise or not, the regularisation strength C = 1.0, the solver, the train/test

ratio and the random seed. The classification threshold (0.3/0.5/0.7) is a decision setting that is applied after training, so it is also chosen by the user and not learned.

3. Influence of the learning rate. The learning rate decides the step size of every gradient update, so it controls the balance between speed and stability (B3). In my experiments 0.001 was too slow to converge in 2000 iterations, 0.1 converged in 58 iterations, 0.5 oscillated before converging, and 1.0 diverged because it is above the limit $2/\lambda max = 0.50$. The range of good values also depends on the scaling of the data (in B2, without standardisation the limit was $\approx 2.7 \cdot 10^{-5}$).

4. Why different metrics? In regression the errors are continuous distances, so MAE, MSE and R^2 measure *how far* the predictions are from the real values (in target units or as explained variance). In classification the outputs are categories, so the question is *how often and in which direction* the model is wrong. For this we use accuracy, precision, recall, F1 and the confusion matrix. Using MSE on 0/1 labels or accuracy on a continuous target would not make sense. Also, accuracy alone can be misleading when the classes are imbalanced and the errors have different costs. In C2, for example, thresholds 0.3 and 0.7 had almost the same accuracy, but the number of missed cancers was very different.

5. Unexpected result. The result that surprised me most was in B3: the models that were stopped early had a slightly *better* test R^2 than the exact least-squares solution (for example 0.455 after 100 iterations vs. 0.453), even though their training loss was higher. At the same time, the s1 coefficient needed thousands of iterations to go from ≈ -7 to its final value of -44. My explanation is this: s1 and s2 are strongly correlated, and this creates a very flat direction in the loss surface (a small eigenvalue). Gradient descent starts from zero, so it first fits the directions that are well defined and only moves slowly along the flat one. When we stop early, these unstable coefficients stay small, and this works like L2 (ridge) regularisation and reduces variance. The difference is small (≈ 0.003 R^2) and with only 89 test patients it could be just noise, but it showed me that the "optimal" training solution is not always the best one on new data. A second, smaller surprise was that lr = 0.5 converged slower than lr = 0.1 even though it is bigger. This happens because it is close to the stability limit and oscillates.

Conclusions

In this lab I saw that linear regression on the Diabetes data gives coefficients that are easy to interpret as partial effects, and a moderate prediction accuracy (test $R^2 \approx 0.45$). Feature scaling does not change the OLS solution, but it is very important for gradient-based training: in my case it reduced the condition number from $\approx 5 \cdot 10^7$ to ≈ 430. The learning rate has to be below the stability limit, and inside that range a medium value converges fastest. Logistic regression separates malignant and benign tumours well (AUC 0.98). Because it gives probabilities, the decision threshold can be changed for the application, and for screening I think a lower threshold (0.3) is better. The remaining errors are along the linear decision boundary where the classes overlap. In my results, adding more informative features (like concave points) helped more than changing the threshold.

References

[1] B. Efron, T. Hastie, I. Johnstone, R. Tibshirani, "Least Angle Regression," *Annals of Statistics*, 32(2), 407–499, 2004. (Diabetes dataset)

[2] W. H. Wolberg, W. N. Street, O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic)," UCI Machine Learning Repository, 1995. https://doi.org/10.24432/C5DW2B

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, 12, 2825–2830, 2011.

[4] G. James, D. Witten, T. Hastie, R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021 (Ch. 3 and 4).

[5] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016 (Sec. 4.3, 7.8 – gradient descent and early stopping).